

Eclipse IDE starter guide

R&D Squad

v1.1

Table of Contents

Eclipse IDE.....	3
Getting started.....	3
Configuration.....	6
File templates.....	7
Formatter.....	9
Name style.....	10
Doxygen.....	13
Other useful settings.....	14
Hints.....	16
Plugins.....	17
Egit.....	17
Mylyn (Redmine).....	20
Eclox (Doxygen).....	21
Project and Files.....	22
New project.....	22
New file.....	25
Project Import.....	27
Paths and Symbols.....	29

Eclipse IDE

Getting started

Starting point and all information can be found here: <http://www.eclipse.org/users/>

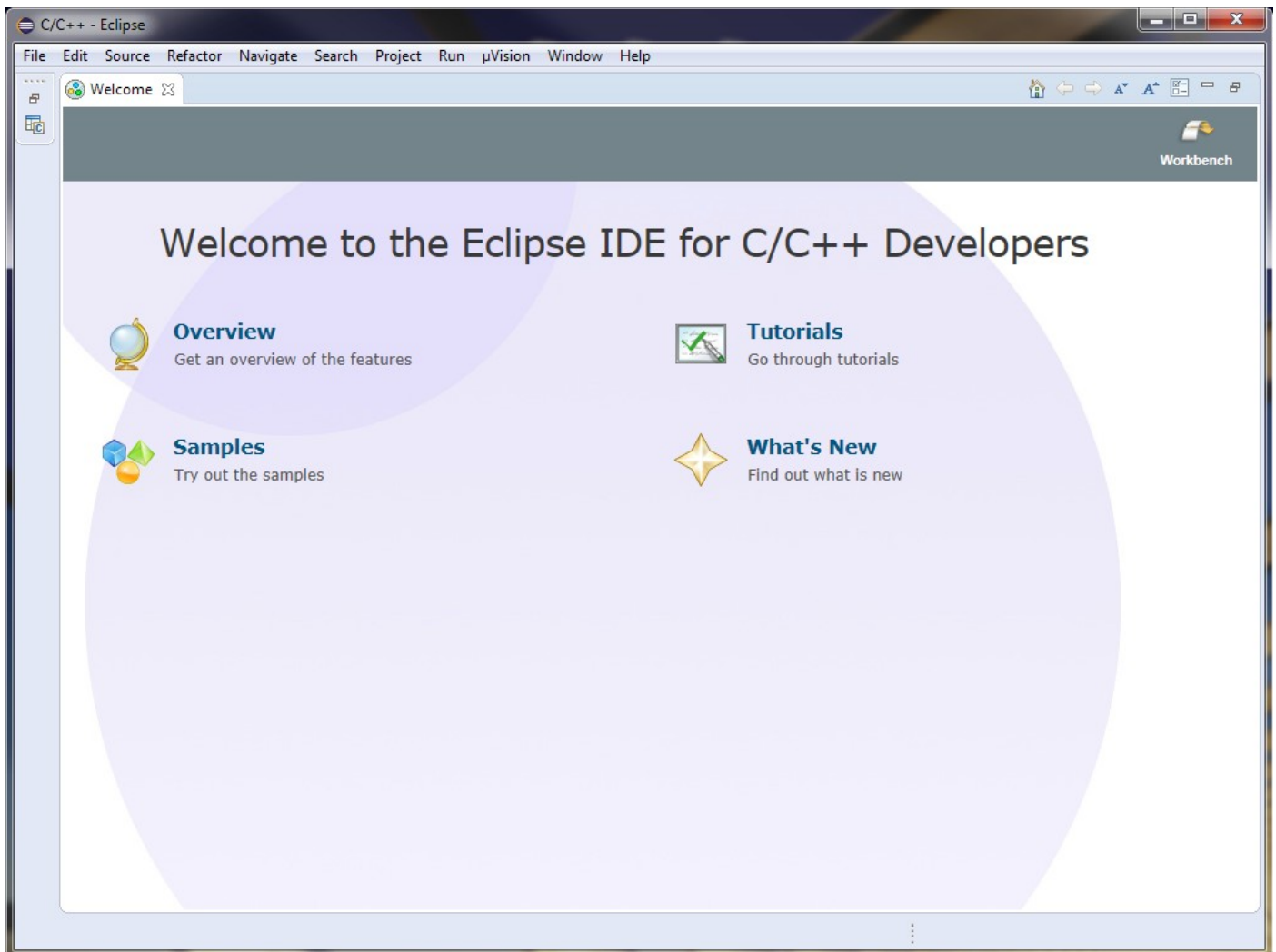
Eclipse download page: <http://www.eclipse.org/downloads/>

Eclipse documentation: <http://help.eclipse.org/mars/index.jsp>

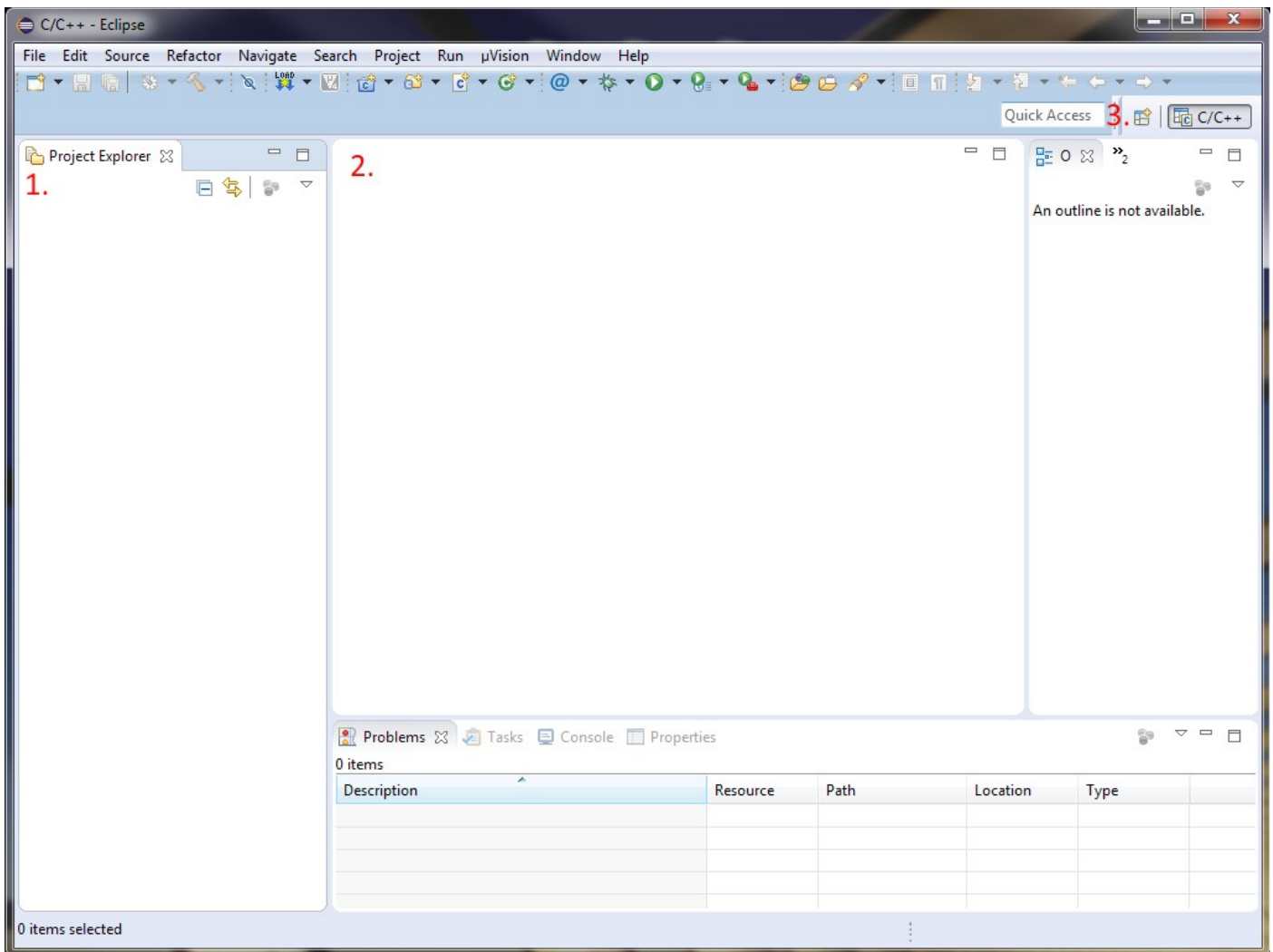
We need Eclipse CDT (version for developing in C/C++). Current version when writing this document is Eclipse CDT MARS.2: <http://www.eclipse.org/downloads/packages/eclipse-ide-cc-developers/mars2>

Eclipse can be installed or used as portable program. We will skip installation procedure thinking that embedded software developers have enough experience to do that. At least I think so...

After installing eclipse and opening it you will be asked to set your workspace (different projects can have different workspace). You will be able to switch between workspace's: *File* → *Switch Workspace*.



Close *Welcome* window or explore it your self.

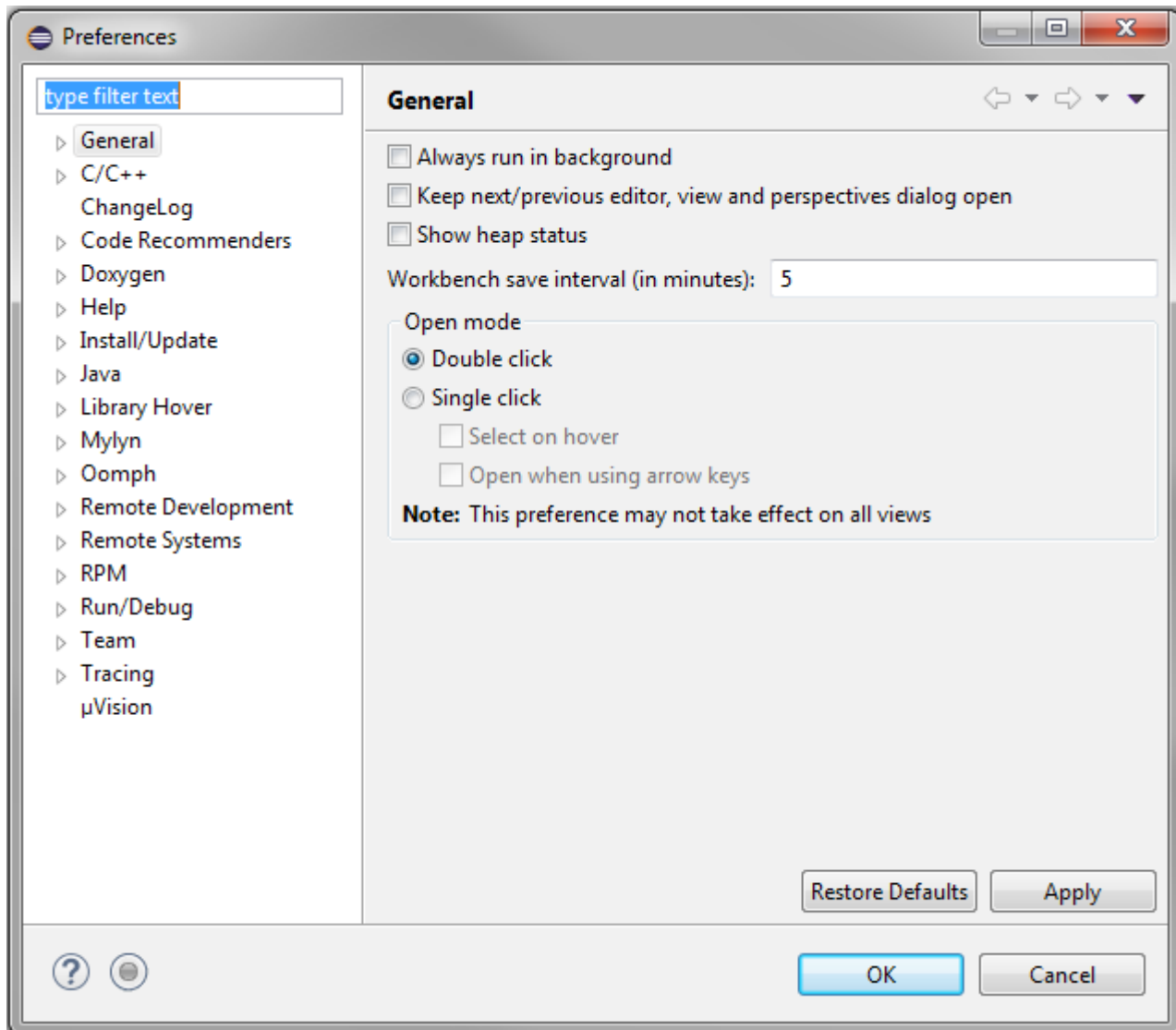


This is your main window:

1. Project explorer – there you will have all your projects.
2. Your editor (nothing opened yet).
3. Perspectives: You can add more by pressing  icon. Most useful are Git and Planing to which we will return in later topics.

Configuration

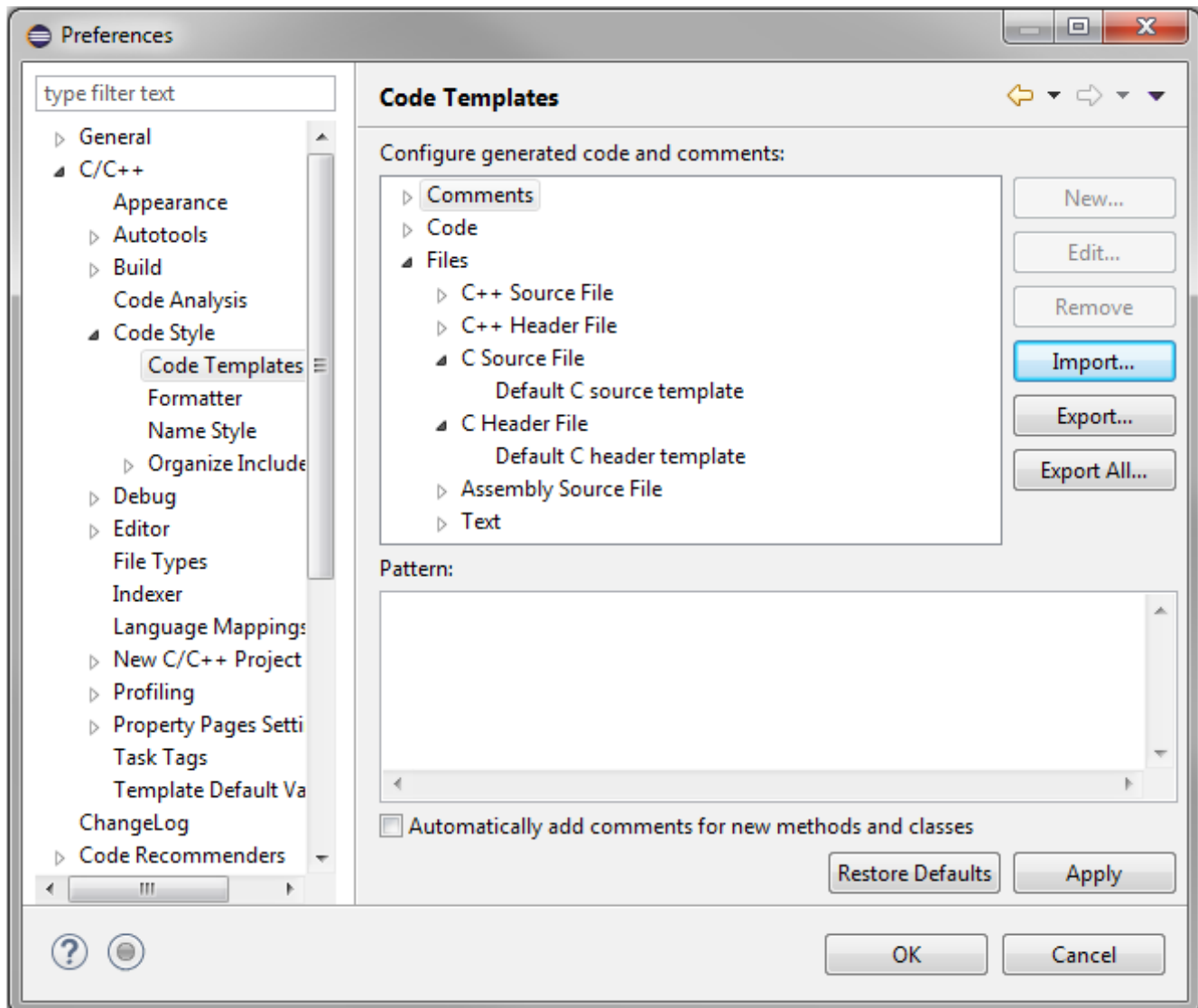
Eclipse configuration can be accessed by going to Window → Preferences.



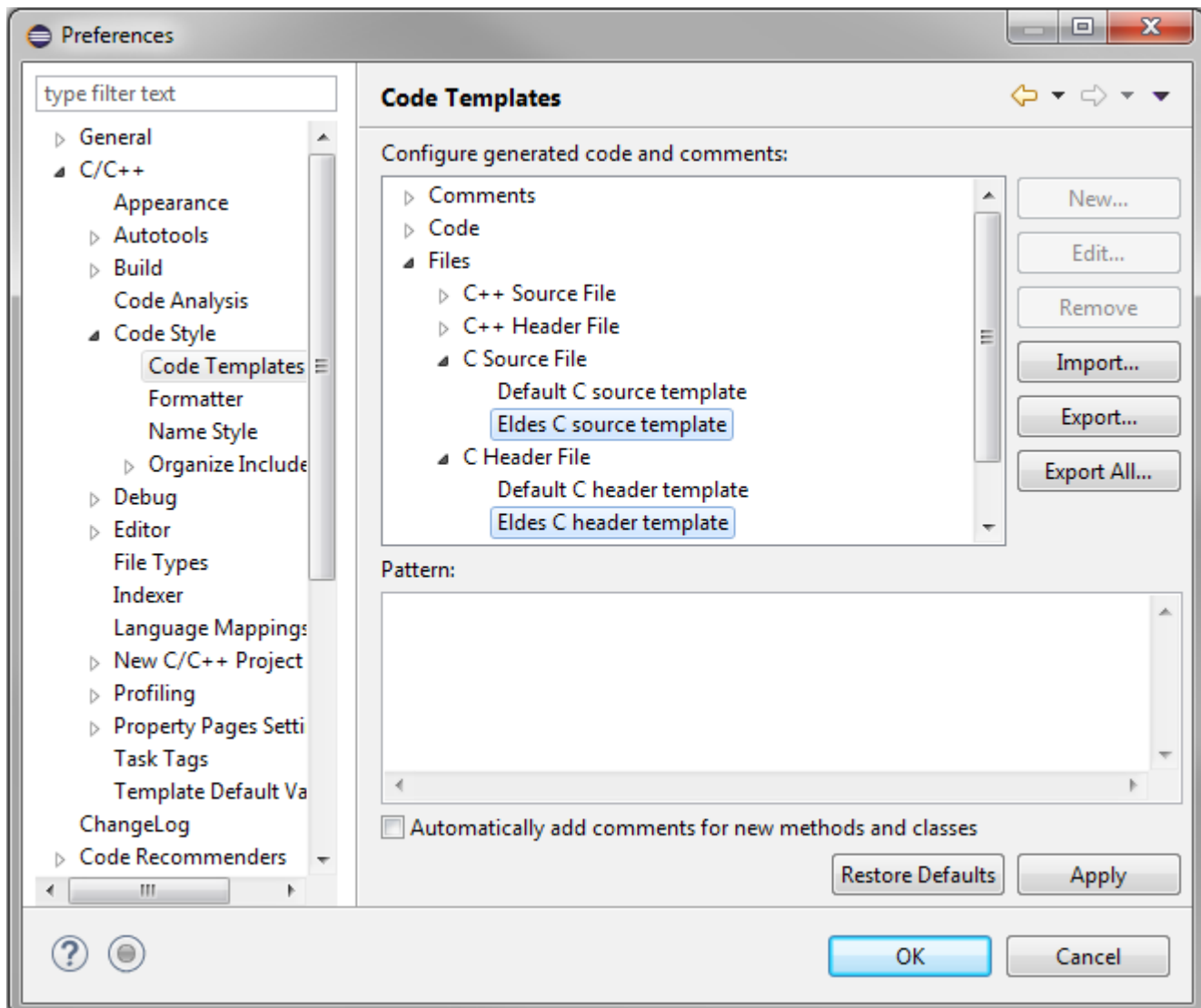
Note: configuration is assigned to workspace.

File templates

File templates are already prepared and should be used as it is. You can import file templates .xml file by going to *Window → Preferences → C/C++ → Code Style → Code Templates* and pressing *Import* button.



File templates should be imported for *C Source File* and *C Header File*.

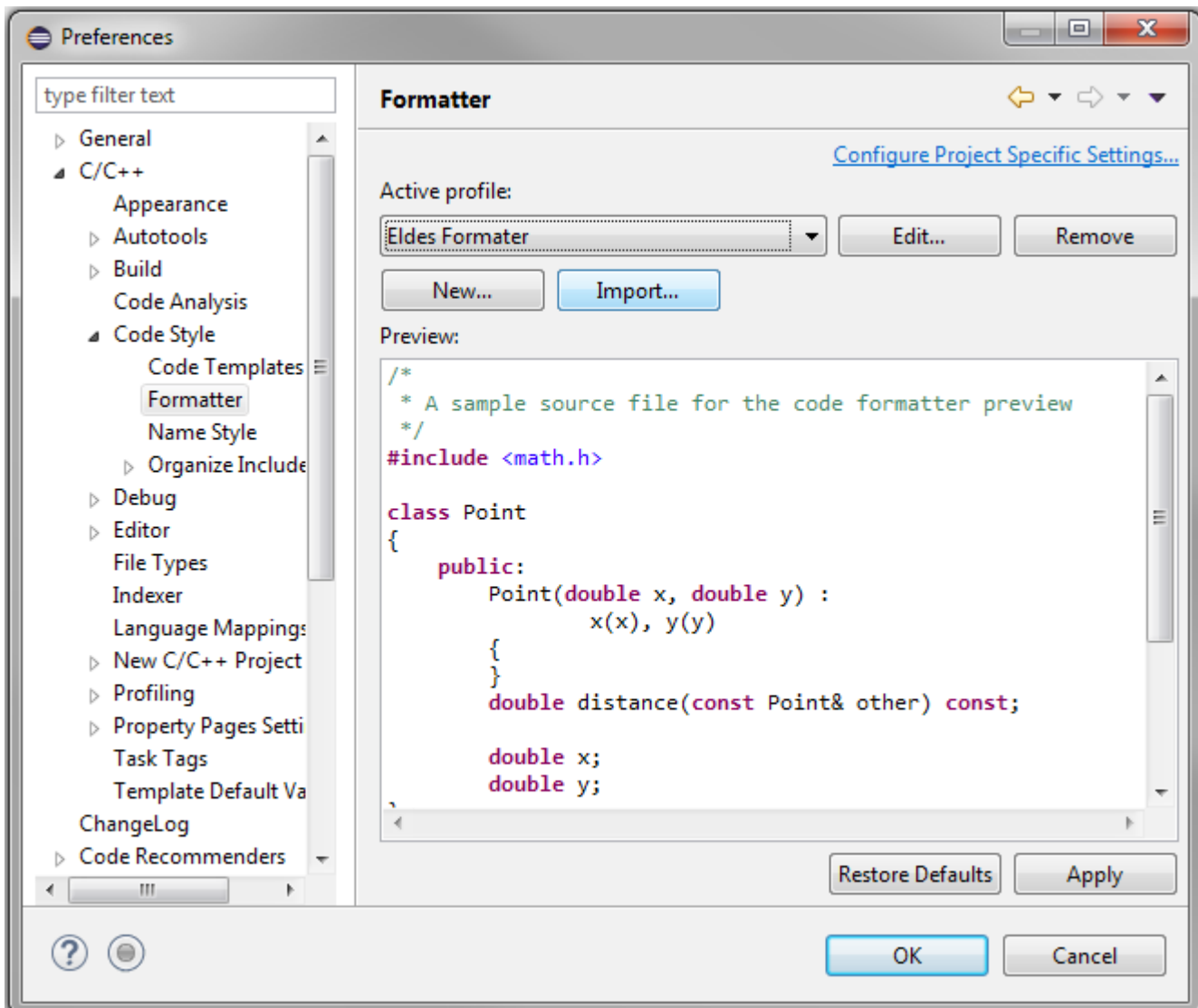


These settings are set by pressing *Apply* button or *OK* (will close *Preferences*).

Formatter

Formatter is used for defining a rule how to format your code. Quick code formatting by defined rules can be managed selecting your code in your editor *CTRL+A* (select your code) and *CTRL+Shift+F*.

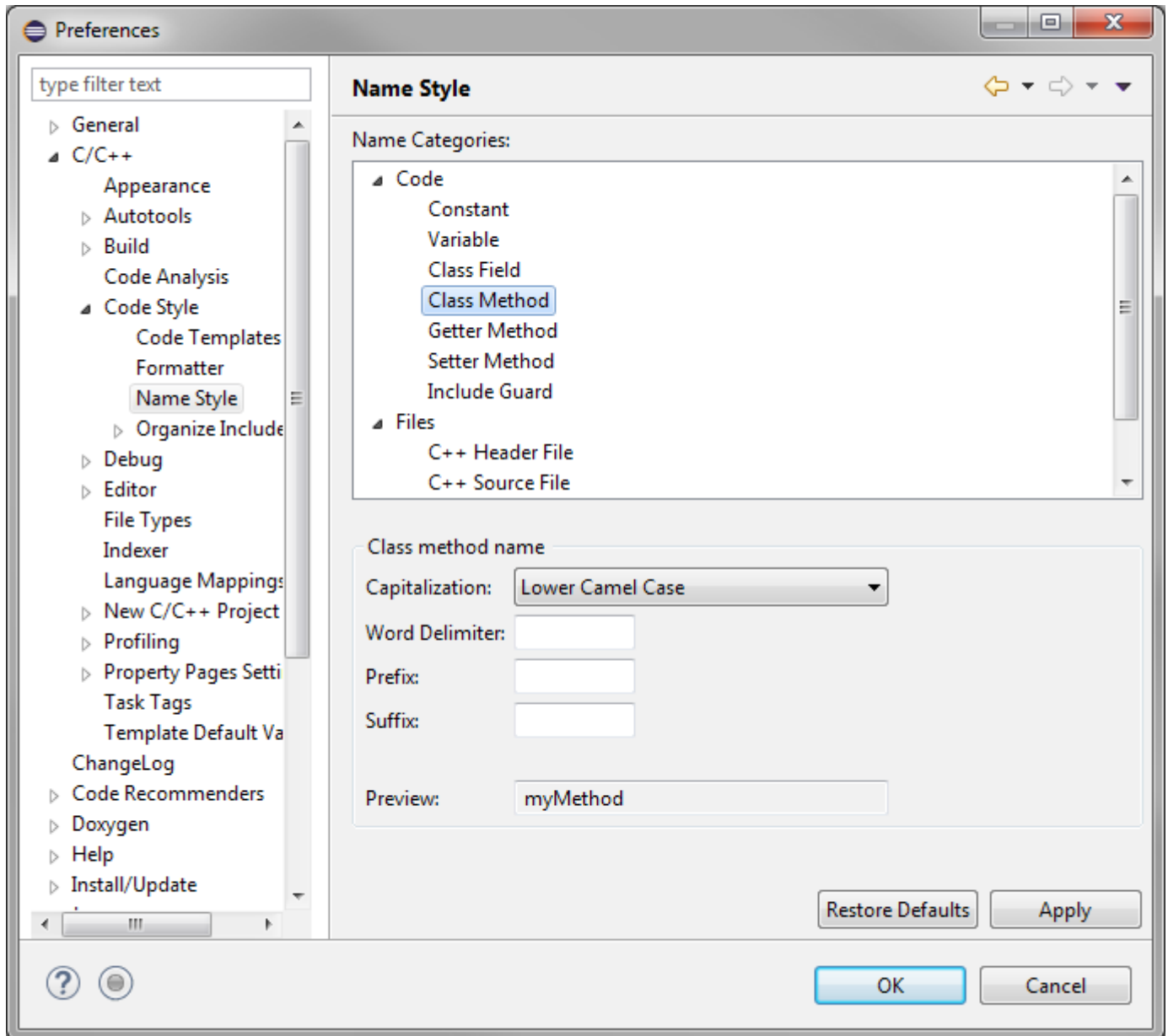
Formatter template (.xml) is prepared and should be used as it is. You should import it by pressing *Import* button.



These settings are set by pressing *Apply* button or *OK* (will close *Preferences*).

Name style

Name styles define rules for how your variables, constants, and etc. should be styled. However, these settings could not be exported or imported and should be set by hand. Please set it as shown below. Every letter or symbol matters.



Code → Constant:

Constant name

Capitalization:

Word Delimiter:

Prefix:

Suffix:

Preview:

Code → Variable:

Variable name

Capitalization:

Word Delimiter:

Prefix:

Suffix:

Preview:

Code → Class Field:

Class field name

Capitalization:

Word Delimiter:

Prefix:

Suffix:

Preview:

Code → Class Method:

Class method name

Capitalization:

Word Delimiter:

Prefix:

Suffix:

Preview:

Code → Getter Method:

Getter name based on the field name

Capitalization:

Word Delimiter:

Prefix: For Boolean:

Suffix:

Preview:

Code → Setter Method:

Setter name based on the field name

Capitalization:

Word Delimiter:

Prefix:

Suffix:

Preview:

Code → Include Guard:

Include guard macro name

☐ File path relative to source folder

☒ File name

☐ Unique identifier

Preview:

Files → C++ Header File:

C++ header file name based on the class name

Capitalization:

Word Delimiter:

Prefix:

Suffix:

Preview:

Files → C++ Source File:

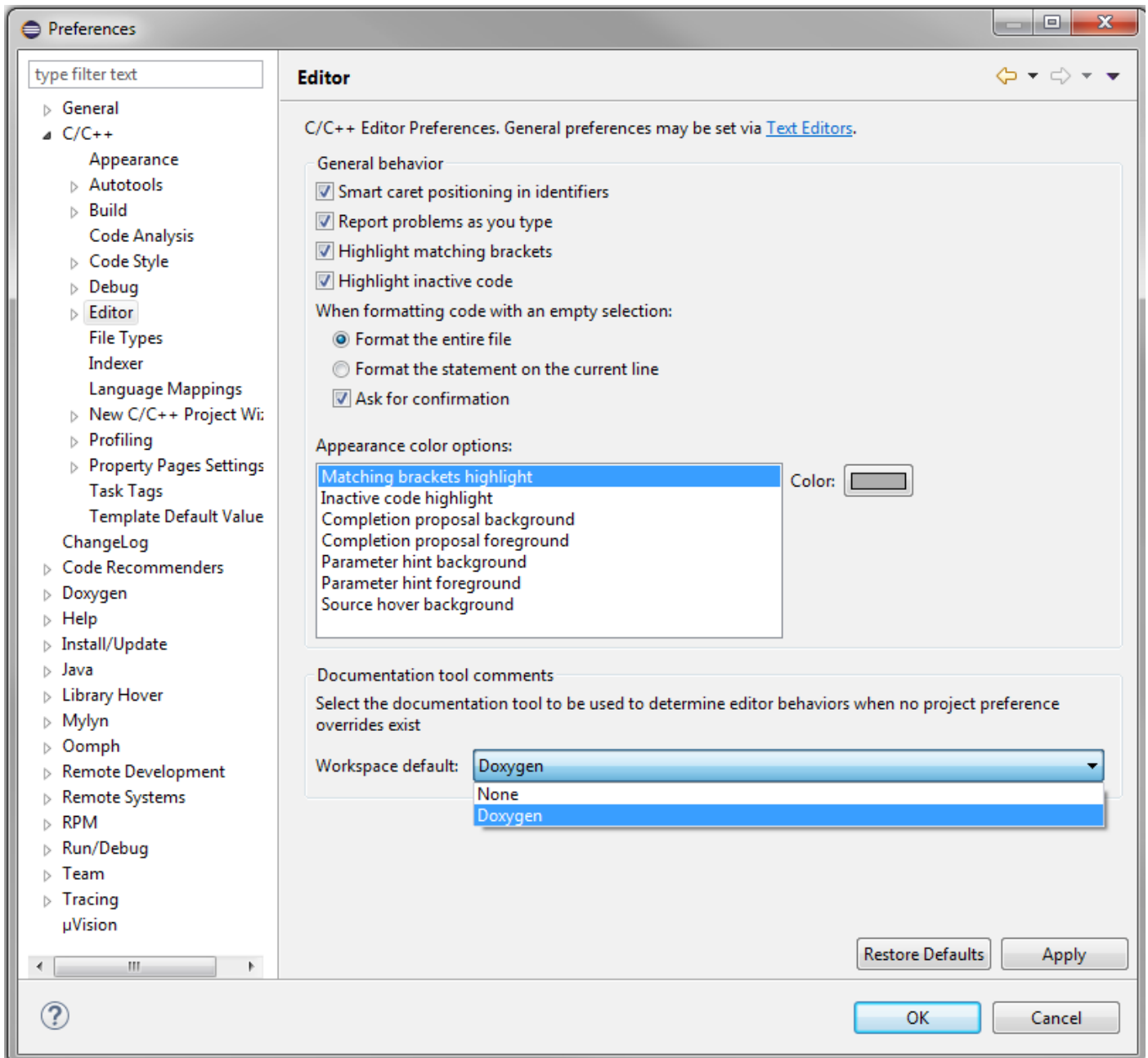
Files → C++ Test File:

C++ source file name based on the class name	C++ test file name based on the class name
Capitalization: Lower Case	Capitalization: Lower Case
Word Delimiter: _	Word Delimiter: _
Prefix:	Prefix:
Suffix: .cpp	Suffix: _test.cpp
Preview: my_class.cpp	Preview: my_class_test.cpp

These settings are set by pressing *Apply* button or *OK* (will close *Preferences*).

Doxygen

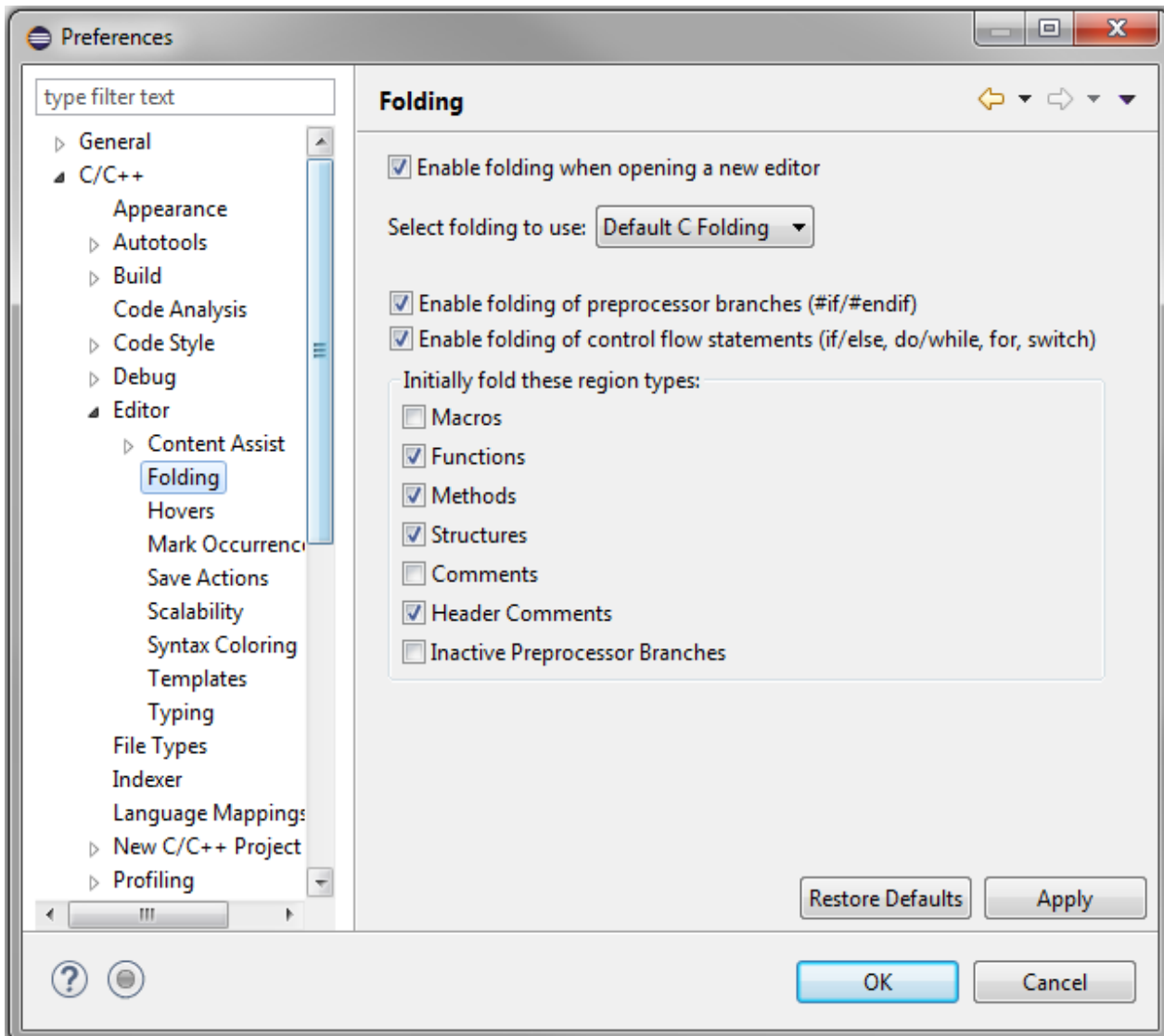
By default editor do not support Doxygen comments, you need to enable it by going to *Window → Preferences → C/C++ → Editor* and in section *Documentation tool comments* setting set *Workspace default* to *Doxygen*.



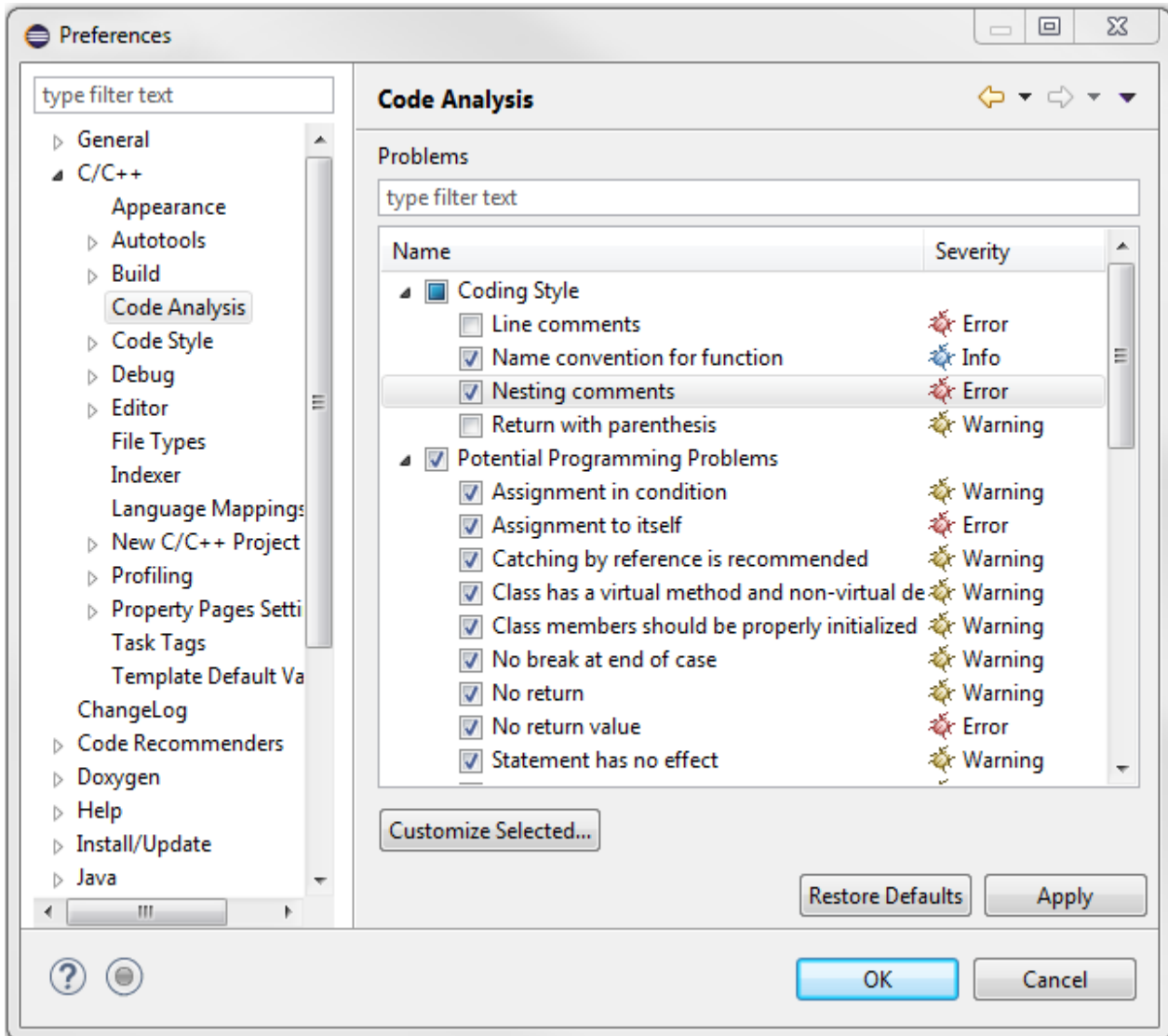
These settings are set by pressing *Apply* button or *OK* (will close *Preferences*).

Other useful settings

Folding: you can set folding settings in *Window* → *Preferences* → *C/C++* → *Editor* → *Folding*:



Code Analysis: set *Name convention for function* and *Nesting Comments* options. This will analyze your code and give you a hints what are you doing wrong. For example, if you name variable or function in wrong style it will give you a warning in exact line where you made a mistake.



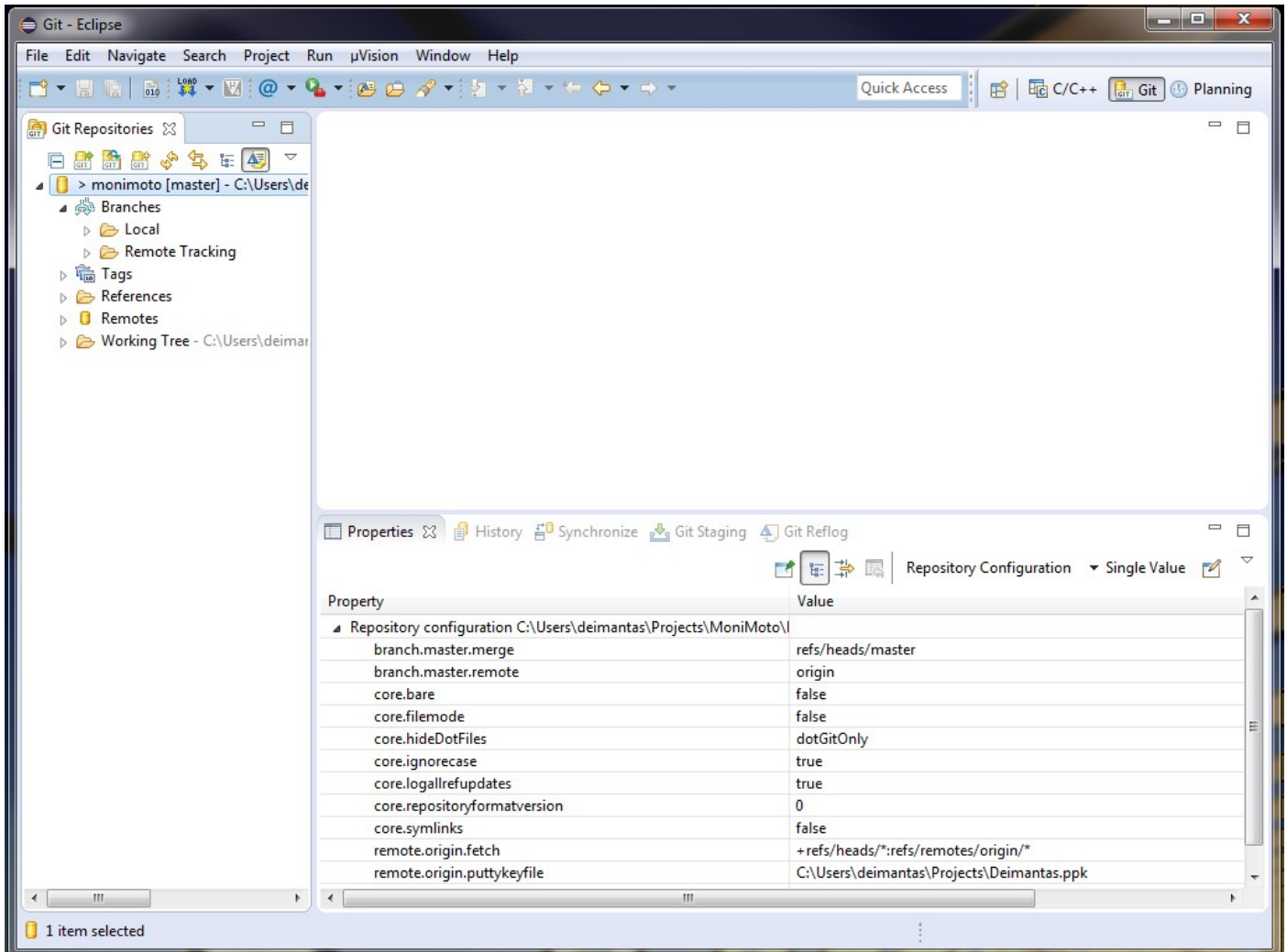
Appearance: for those who like dark themes, you can go to *Preferences* → *General* → *Appearance* and change a theme to *Dark* or what ever you like. After changing a theme sometimes it's best to reopen the eclipse. Your specific designer needs like colors and fonts can be managed by going to *Preferences* → *General* → *Appearance* → *Colors and Fonts*.

Hints

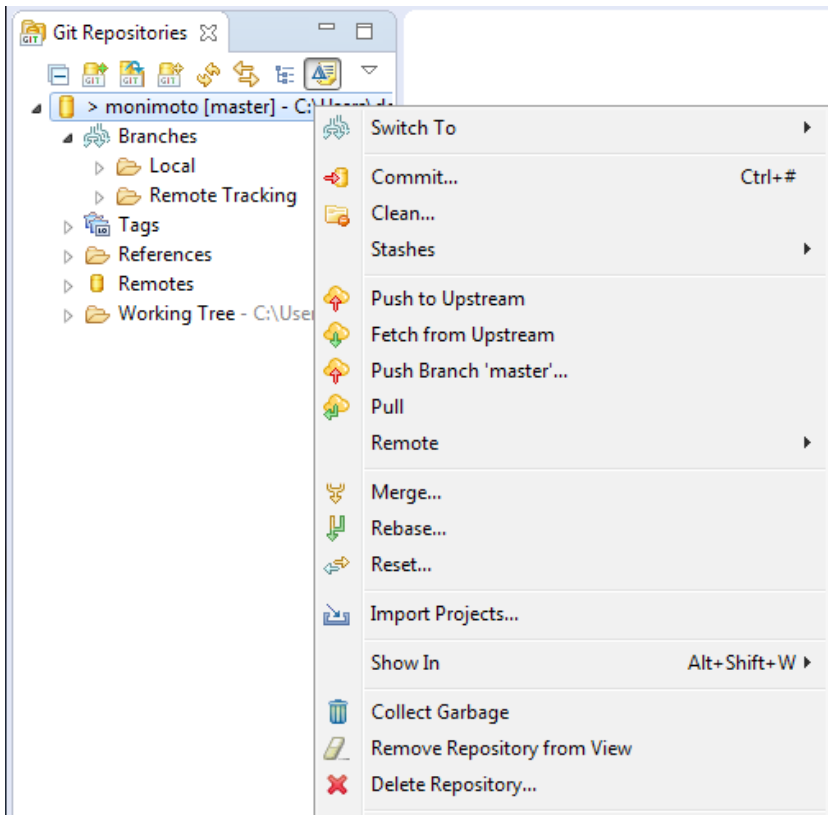
Plugins

Egit

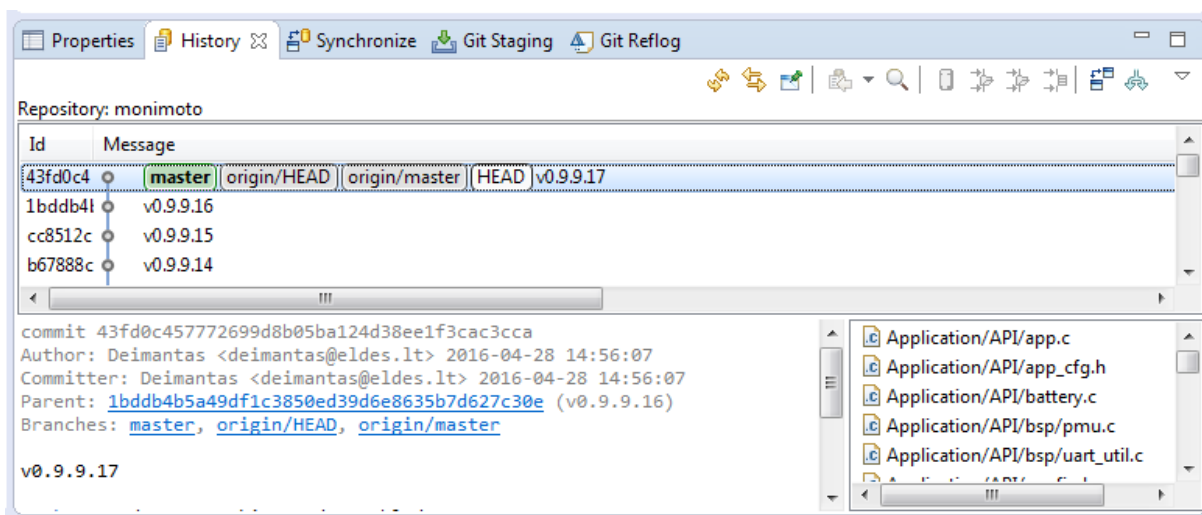
Egit is a Git plugin for eclipse which is already pre'installed. You can access and manage your project repositories by Opening Git perspective.



Git command like Push, Pull and etc can accessed by pressing right mouse button on your Git repository.



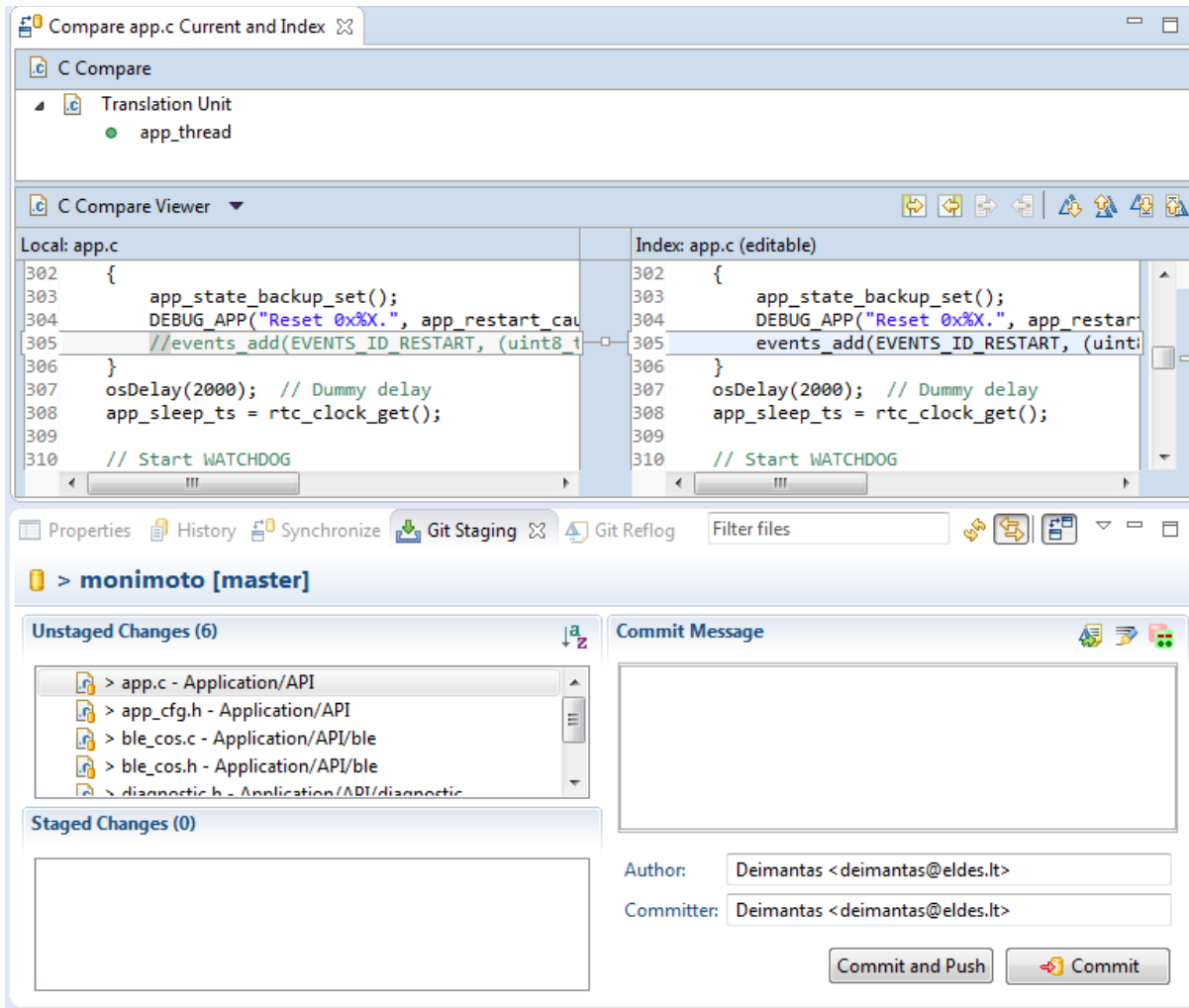
There are few more useful windows like *History*, *Git Staging*, *Git Reflog*.



In history Tab you can list Git repository history (commits), by pressing right mouse button on commit you can access more commands.

Git staging window is useful when making a commit. It shows all your changed files and by opening that file you can view changes in it compared remote file.

It is very useful before committing your changes to go through all your changed files and check your changes again.



To confirm that file is ready to commit you should add to index it by pressing right mouse and selecting “Add to Index”.

In *Commit Message* window you can write your commit message and “Commit” your changes or “Commit and Push”.

Mylyn (Redmine)

Bla bla bla...

Eclox (Doxygen)

Installation

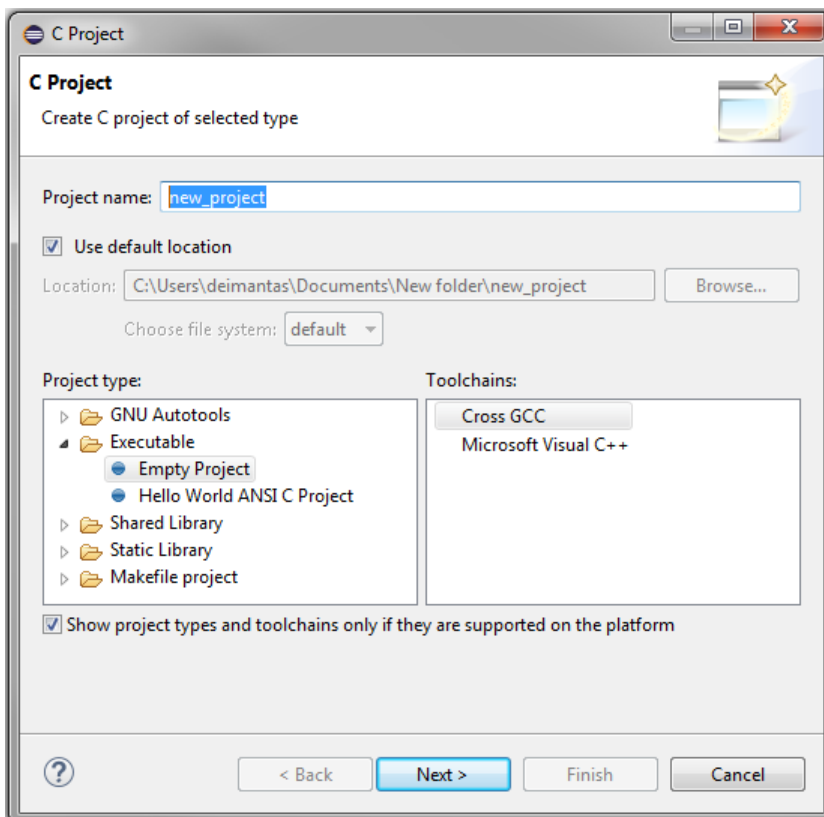
Creating

Generating

Project and Files

New project

Create your project by pressing *File* → *New* → *C Project* or by pressing right mouse button in your Project Explorer and selecting *New* → *C Project*.



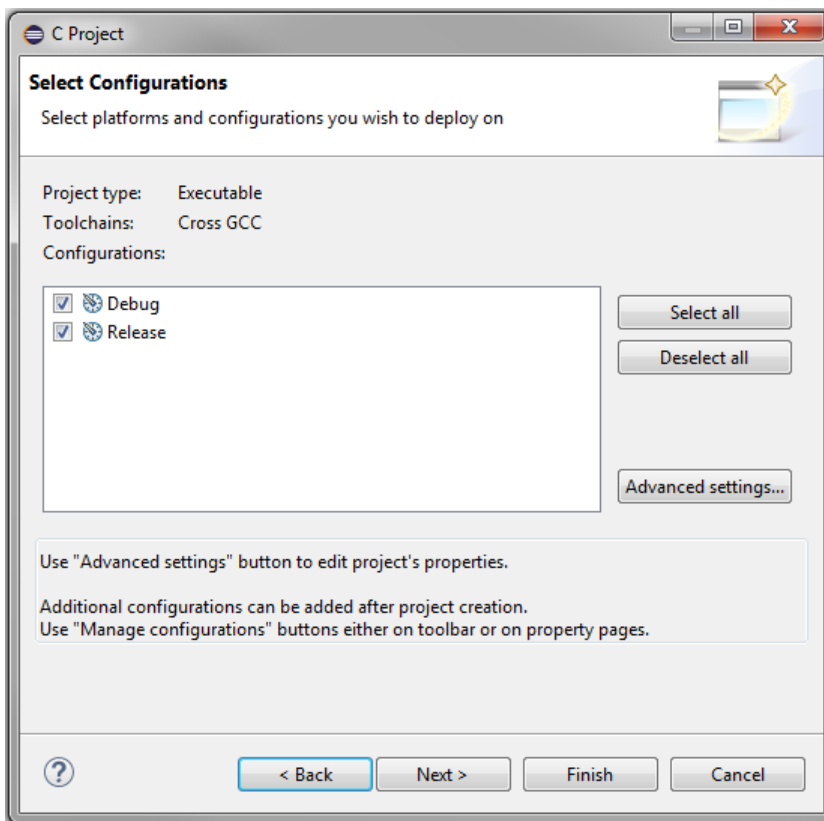
Project name: enter your project name.

Use default location: if project should be located in other location please unmark and specify new one by pressing “Browse...” button.

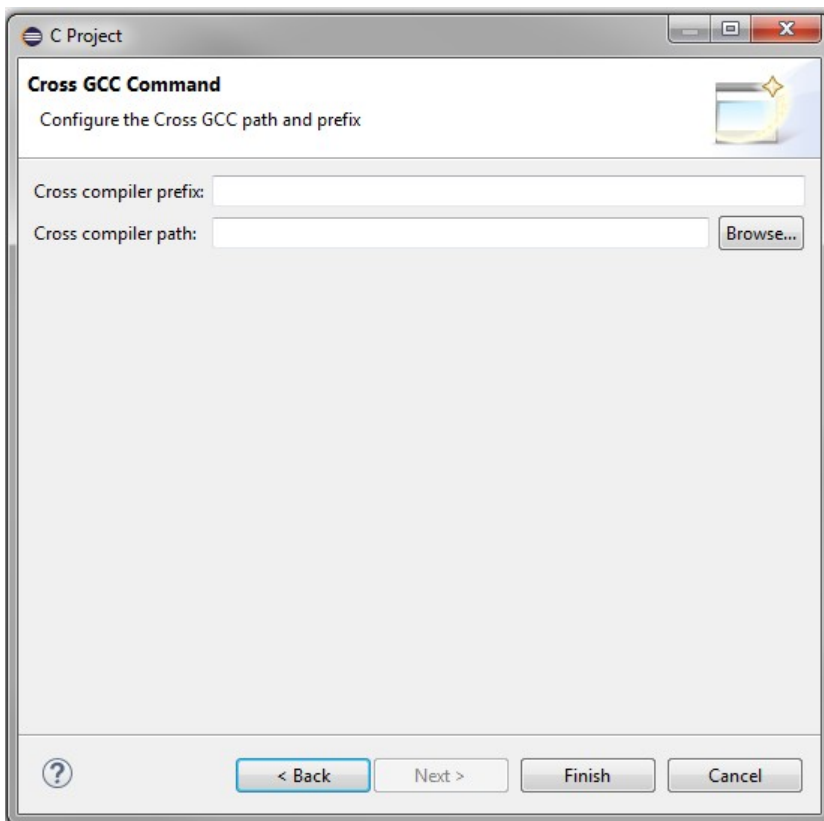
Project type: project type should be “Empty Project”.

Toolchains: Select “Cross GCC” (we won't use it).

Press “Next >” button.

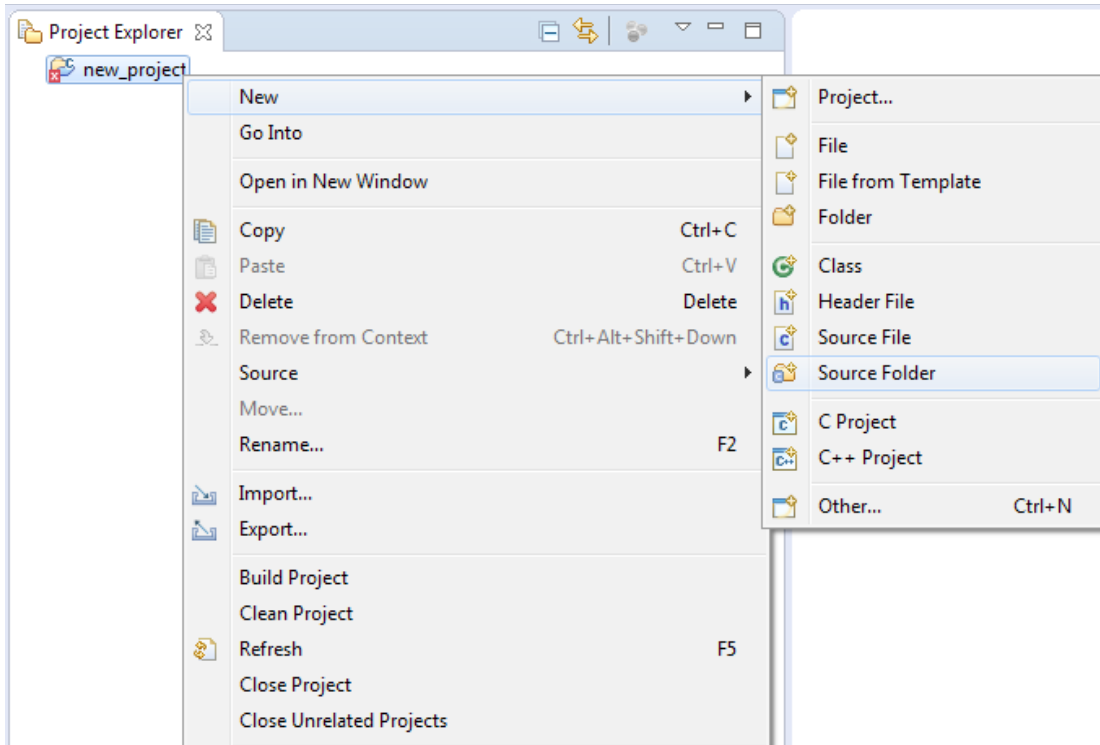


Press “Next >” button.



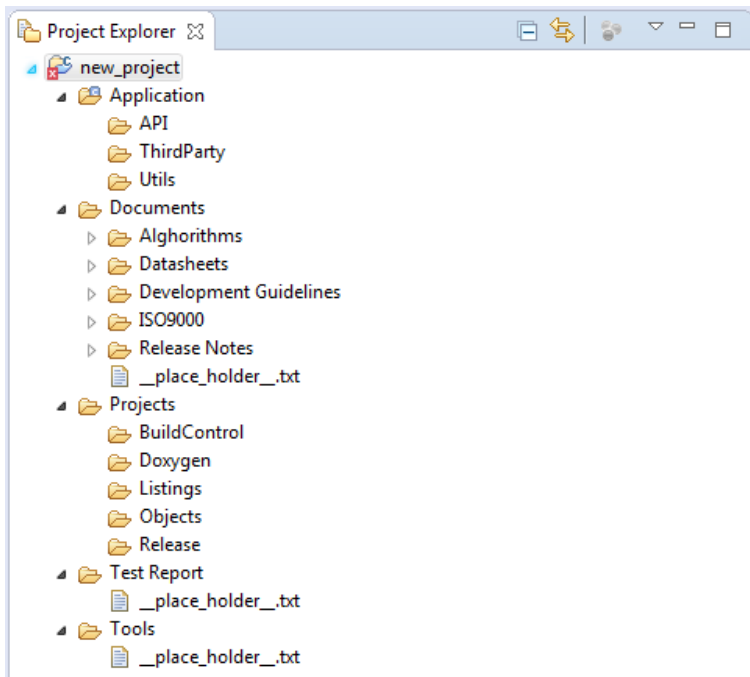
Press “Finish” button.

New folders or files can be created by pressing right mouse button on desired project or project folder.



Application folder should be created as a source folder.

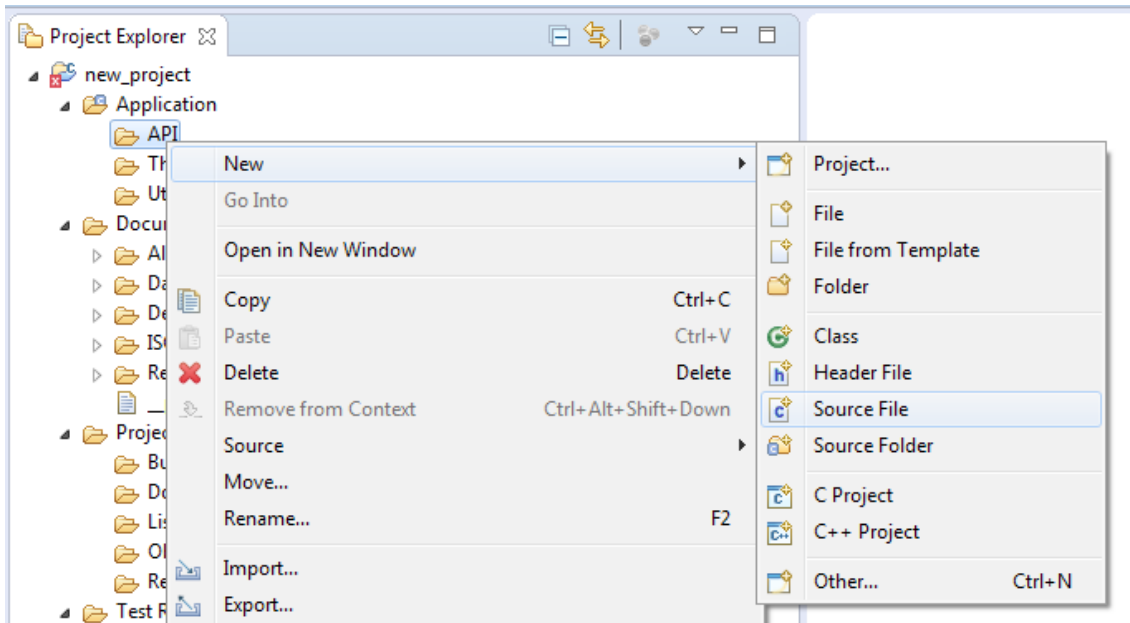
Project structure should be managed by hand, creating new folders and files one by one as defined in Project structure document.



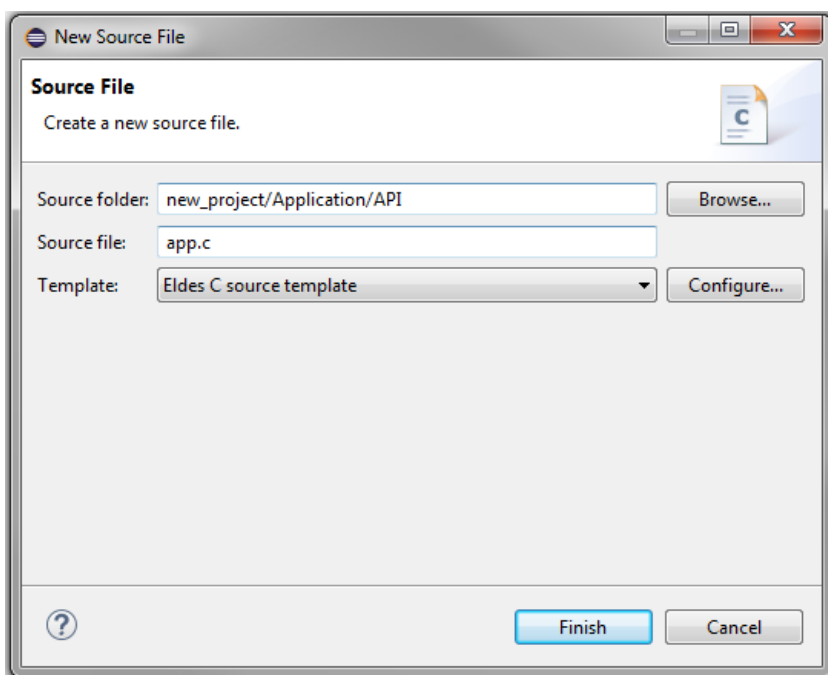
New file

Source file

New C source file can be created by pressing right mouse button on desired project folder (where you would like to create a file) and selecting *New* → *Source File*.



Fill up all fields and press “*Finish*” button.



Source Folder: creating new C Source file you should specify where file should be created.

Source file: C source file name with extension “.c”.

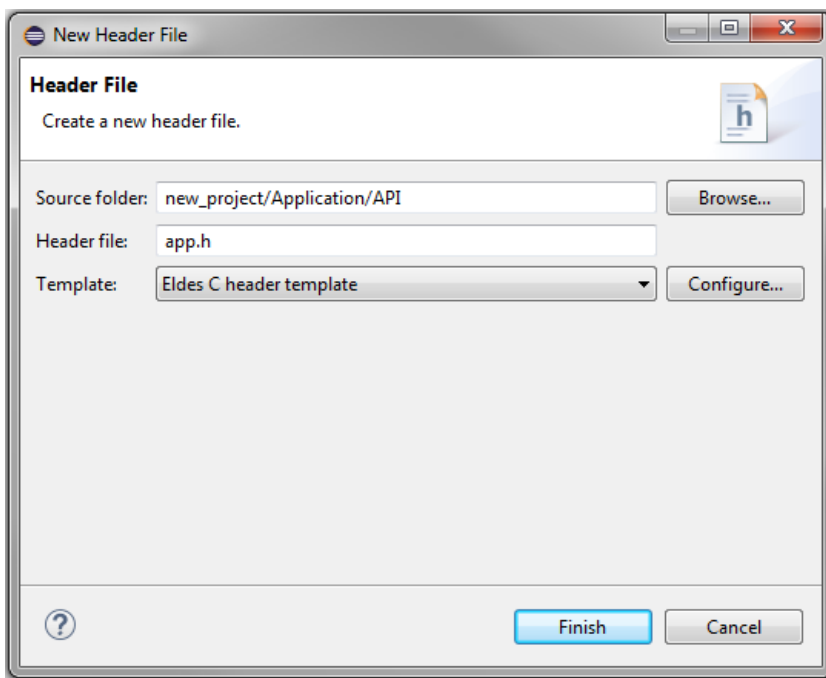
Template: Select “*Eldes C source template*”. How to import file templates go to topic “*Configuration/File*”

templates”.

Header file

New C header file can be created by pressing right mouse button on desired project folder (where you would like to create a file) and selecting *New → Header File*.

Fill up all fields and press “*Finish*” button.



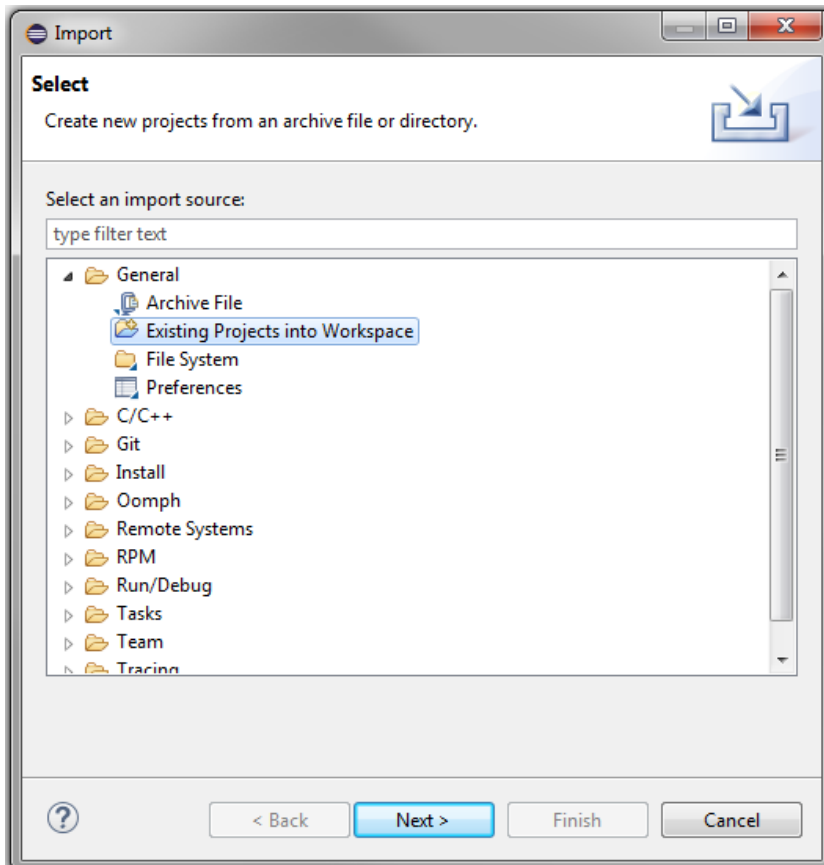
Source Folder: creating new C header file you should specify where file should be created.

Source file: C header file name with extension “.h”.

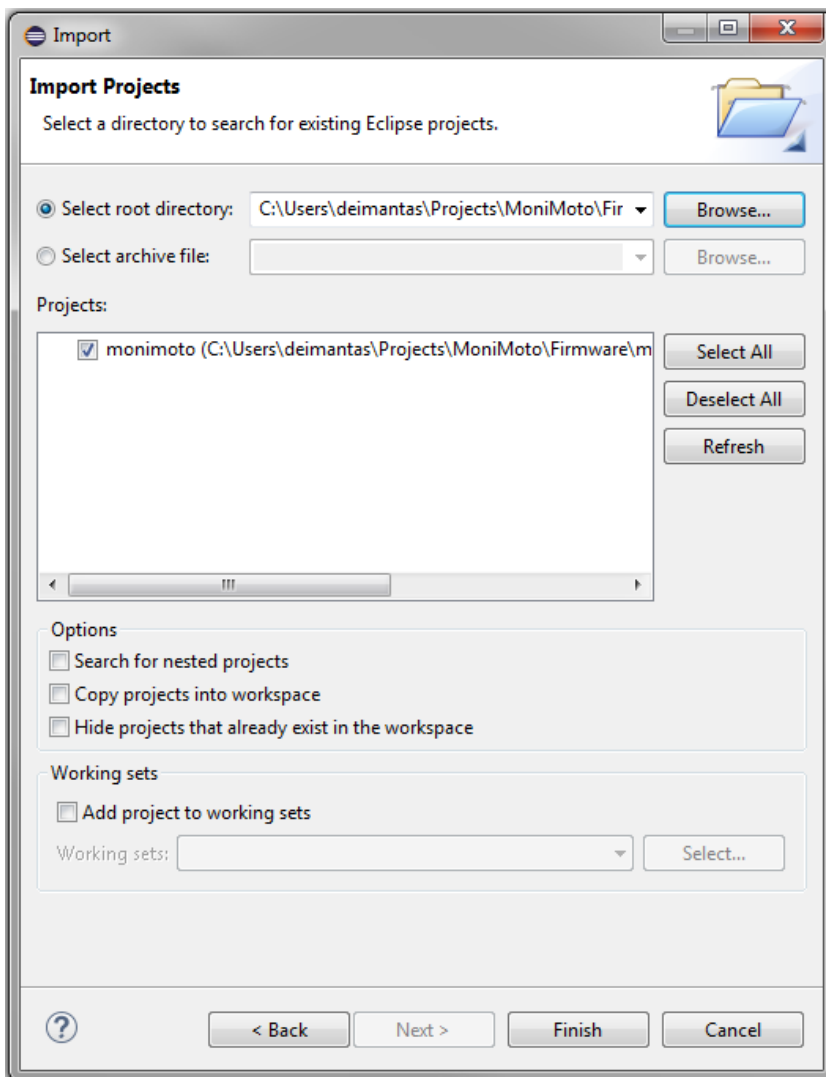
Template: Select “*Eldes C header template*”. How to import file templates go to topic “*Configuration/File templates*”.

Project Import

Existing project can be imported into workspace by going to *File* → *Import* or by pressing right mouse button in you Project Explorer and selecting “*Import*”.

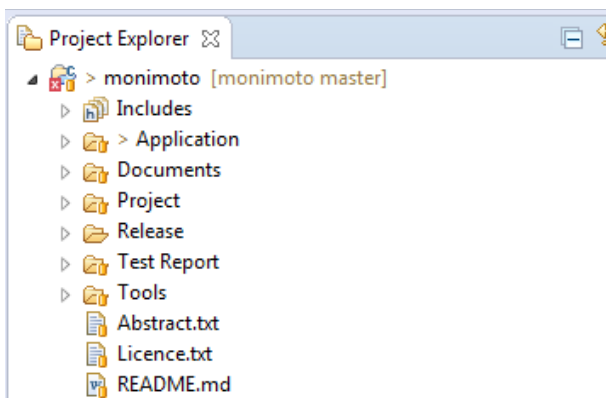


Select “*General* → *Existing Projects into Workspace*” and press “*Next >*”.



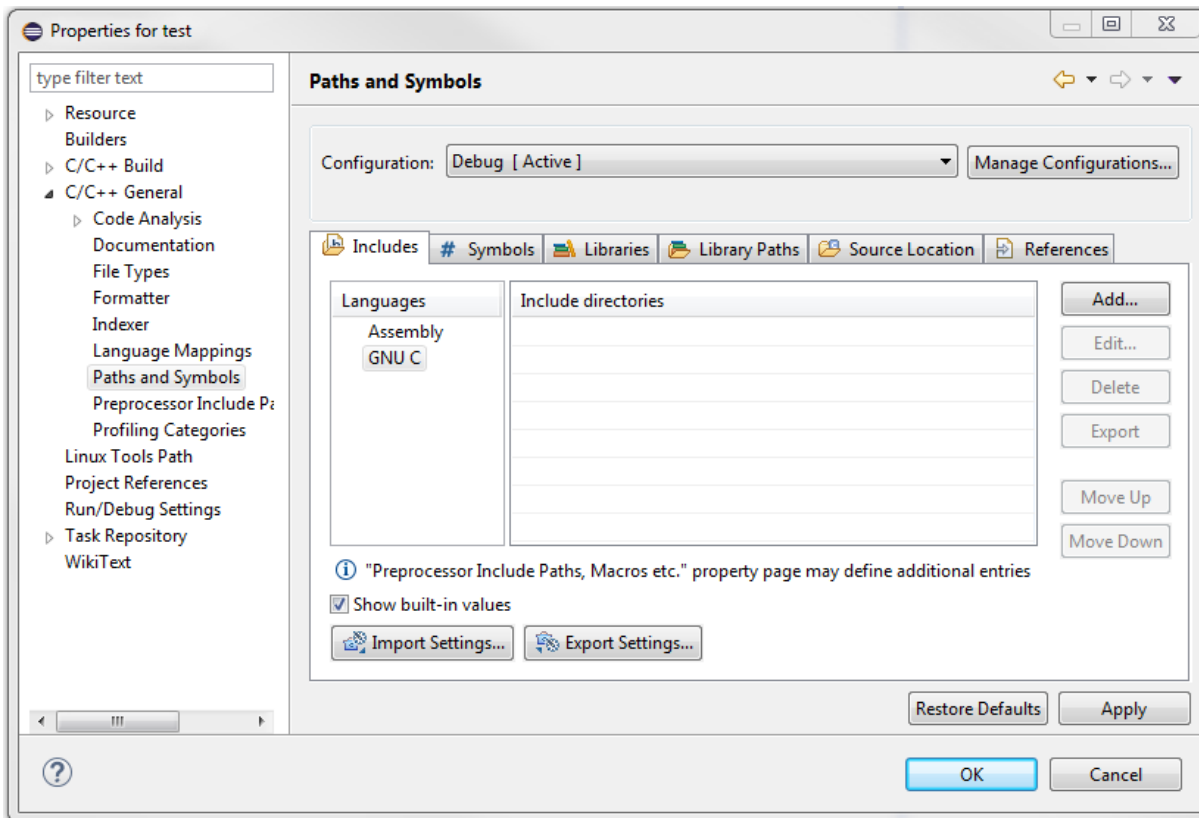
Select root directory where your project which you would like to import is located.

Mark a project and press “*Finish*” and your project after couple of moments will be imported into your workspace.



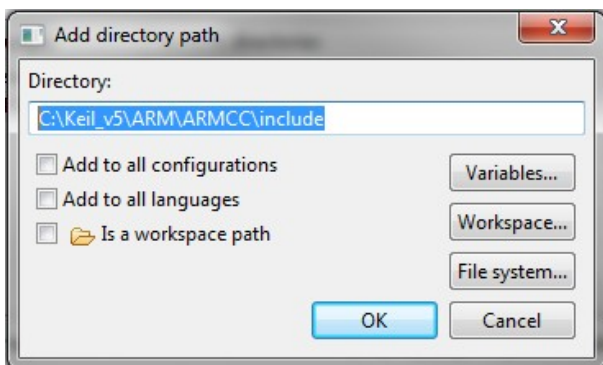
Paths and Symbols

You can access your project *Paths and Symbols* by pressing right mouse button on your project in “*Project Explorer*” and choosing “*Properties*”. In Properties window expand “*C/C++ General*” and select “*Paths and Symbols*”.

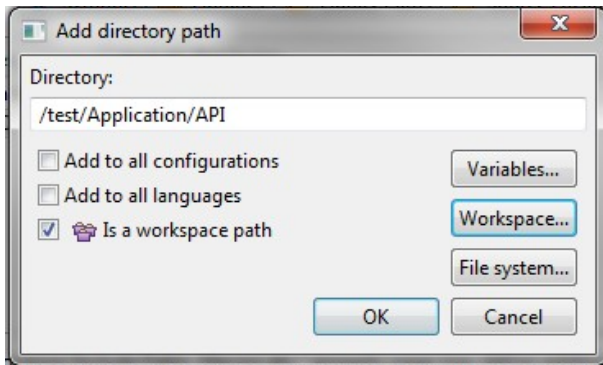


Select language “*Assembly*” or “*GNU C*” depending on what language you need to update “*Paths and Symbols*”. In this example we will add *Includes* for *GNU C* language.

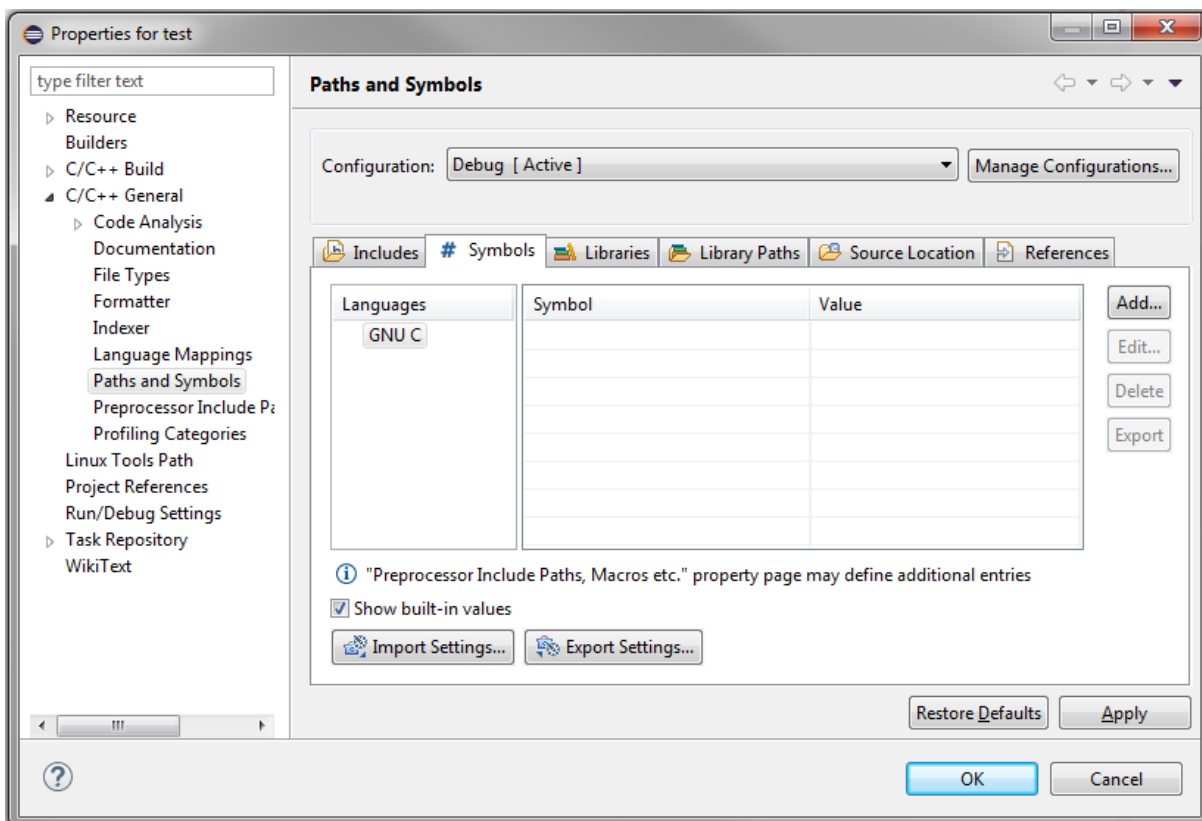
First include path you always need to add is compiler includes, in this example it is “*ARMCC*” compiler includes. Press “*Add*” button, in window “*Add directory path*” press “*File system..*” and navigate to folder “*C:\Keil_v5\ARM\ARMCC\include*” (in your system this folder may be different). Press “*OK*” and your include path will be added to your project.



Next include path you will need to add will be your project includes. Press “Add” button, in opened window “Add directory path” press “Workspace..”, in opened window navigate to your projects directory which has to be added as include path and press “OK”.



To add a symbols to your project configuration, go to tab “Symbols” and press “Add” button.



In opened window “Add symbol” type your symbol name and value if necessary and press “OK” button. For example we need to add symbol (definition) to use ARM core M0.

